30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 2

Fragments, FragmentManager & Navigation — in a Compose World



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 2

Why Fragments are still Day 2 in 2026

New screens are usually written in Compose — yet interviews still ask about Fragments. Why:

- Most production apps still have **years of Fragment code**
- Fragments are **not deprecated**, and AndroidX keeps shipping them (predictive back, Compose interop)
- Real work in 2026 is **interop and migration**: Compose inside Fragments, Fragments inside Compose
- Fragment bugs — leaks, state loss, back stack — are classic senior filters

Level	Questions
Junior	Two lifecycles · Fragment vs Activity vs Composable · Communication
Mid	State & process death · Transactions · Back stack & back handling · Pitfalls
Senior	Navigation design · Smooth transitions · Compose interop & migration
Architect	Multi-team navigation · Scoped state · ANRs & responsiveness



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: two lifecycles

A Fragment has **its own lifecycle** and a **separate, shorter view lifecycle**:

```
Fragment: CREATED ──── STARTED ─ RESUMED ─ … ─ DESTROYED
View:         onCreateView ──────── onDestroyView
              (view exists)         (view gone, Fragment
                                     may still be alive)
```

- A Fragment on the back stack has **no view** but is still alive (CREATED)
- The view can be destroyed and recreated **many times** for one Fragment instance
- Anything that touches views must follow `viewLifecycleOwner`, not the Fragment

THE RULE BEHIND HALF OF ALL FRAGMENT BUGS

The Fragment outlives its view. Every leak and "view is null" crash in this deck comes back to that sentence.



Esmaeeil Moradi
@esmaeeil-moradi700

Junior

Two lifecycles · Fragment vs Activity vs Composable · Communication



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. The Fragment lifecycle and its host

```
onAttach → onCreate → onCreateView → onViewCreated
→ onViewStateRestored → onStart → onResume
… onPause → onStop → onSaveInstanceState*
→ onDestroyView → onDestroy → onDetach
```

Callback	Use it for
`onCreate`	Read `arguments`, obtain the ViewModel. **No views yet**
`onCreateView`	Inflate or return a `ComposeView` — nothing else
`onViewCreated`	Bind views, start collecting with `viewLifecycleOwner`
`onDestroyView`	Clear view references (binding, adapters holding views)
`onDestroy` / `onDetach`	Instance gone / detached from host

* `onSaveInstanceState` can run any time before `onDestroy` — don't assume an order.



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Rules interviewers check

- A Fragment's state can **never exceed its host's**: host STARTED → its Fragments are at most STARTED.
- `setMaxLifecycle` can cap it further — `ViewPager2` keeps off-screen pages at STARTED, so only the visible page is RESUMED.
- `onActivityCreated` is **deprecated**. Use `onViewCreated` for view setup, or observe the activity's lifecycle.

INTERVIEW TRAP

"Pass data through the Fragment's constructor." — **No.** After process death the system recreates the Fragment with its **no-arg constructor**, so constructor arguments are lost. Use `arguments` (or type-safe navigation routes). Constructor injection needs a `FragmentFactory`.

FOLLOW-UP

"When is `viewLifecycleOwner` not available?" — Before `onCreateView` returns and after `onDestroyView`. Accessing it then throws.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Activity, Fragment or Composable?

Use...	When
Activity	Real entry points: launcher, deep links, share targets, picture-in-picture, a separate task
Fragment	Existing View-based screens, incremental migration, libraries that still require them
Composable destination	New screens in 2026 — single activity + Compose Navigation

INTERVIEW TRAP

"Fragments are deprecated." — They are **not**. Google recommends Compose for new UI, but Fragments remain supported and receive new features.

INTERVIEW TRAP

"We use Fragments because Activities are heavy." — Weak answer. The real benefits of a single activity are **shared scope** (ViewModels, DI), **one navigation graph** and **controlled transitions**, not raw memory.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. How do Fragments communicate?

Need	Modern tool
Shared, ongoing state	**Shared ViewModel** at the right scope: activity, nav graph or parent Fragment
One-off result to the previous screen	**Fragment Result API**, or the Navigation back-stack entry's `SavedStateHandle`
Arguments to the next screen	`arguments` / type-safe routes
Talk to the host activity	Shared ViewModel — avoid casting `requireActivity()` to an interface

```
// Receiver (e.g. ProfileFragment)
setFragmentResultListener(REQUEST_PICK_CITY) { _, bundle ->
    viewModel.onCitySelected(bundle.getString(KEY_CITY))
}

// Sender (e.g. CityPickerFragment)
setFragmentResult(REQUEST_PICK_CITY, bundleOf(KEY_CITY to city))
findNavController().popBackStack()
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. The details that matter

- Fragment results are delivered only when the listener is **at least STARTED**, and only the **latest** result per key is kept.
- Results survive configuration changes and process death — they're part of `FragmentManager` saved state.
- With Navigation:

```
// Sender
findNavController().previousBackStackEntry
    ?.savedStateHandle?.set(KEY_CITY, city)

// Receiver
findNavController().currentBackStackEntry
    ?.savedStateHandle?.getStateFlow<String?>(KEY_CITY, null)
```

INTERVIEW TRAP

Never hold a reference to another Fragment (`findFragmentByTag(...)` then call its methods). It may be destroyed, recreated, or have no view — and it couples two screens forever.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 2

Mid-Level

State & process death · Transactions · Back stack · Pitfalls



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Fragment state: rotation and process death

State	Where it lives
Screen arguments	`arguments` Bundle — also the ViewModel's `SavedStateHandle` defaults
Keys to rebuild the screen	`SavedStateHandle`
View state (text, scroll, checked)	Saved automatically for views **with an ID**
Screen data	ViewModel (memory) + repository (disk / network)

- `setRetainInstance(true)` is **deprecated** — a ViewModel covers the same need without retaining the Fragment.
- The back stack itself — and the saved state of every Fragment on it — is restored by `FragmentManager` after process death.

INTERVIEW TRAP

"Fragment saved state doesn't survive process death." — **Wrong**, same as for activities: it survives system-initiated process death and is lost only when the user dismisses the task.



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Test it the right way

Technique	Simulates
`FragmentScenario.recreate()`	Configuration change
Background → `adb shell am kill <package>` → reopen from Recents	**Process death**
"Don't keep activities"	Activity destruction — **process stays alive**

```kotlin
@Test fun keepsQueryAfterRecreate() {
    val scenario = launchFragmentInContainer<SearchFragment>()
    onView(withId(R.id.query)).perform(replaceText("kotlin"))

    scenario.recreate()

    onView(withId(R.id.query)).check(matches(withText("kotlin")))
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Transactions: what each operation does

Operation	Old Fragment	Its view
`add(B)` on top of A	A stays **RESUMED**	A's view stays (underneath)
`replace(B)` + back stack	A → **CREATED**	Destroyed (`onDestroyView`)
`replace(B)` without back stack	A → **DESTROYED**	Destroyed
`hide(A)` / `show(A)`	Lifecycle **unchanged**	`GONE` / visible
`detach(A)` / `attach(A)`	CREATED, still managed	Destroyed / recreated

INTERVIEW TRAP

"*`add()` pauses the Fragment below.*" — **No.** Both Fragments are RESUMED, both receive clicks unless the top one consumes them, and both keep their views in memory. That's why `replace` is the default.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Committing safely

```
supportFragmentManager.commit {
    setReorderingAllowed(true)
    replace<DetailFragment>(
        R.id.container, args = bundleOf("id" to id)
    )
    addToBackStack("detail")
}
```

- Always `setReorderingAllowed(true)` — it's required for postponed transitions and avoids redundant lifecycle work.
- `commit()` is **asynchronous**; `commitNow()` is synchronous but **can't** be added to the back stack.
- Host in a **`FragmentContainerView`**, not a `FrameLayout`: it fixes animation z-ordering and dispatches window insets.

INTERVIEW TRAP

Committing after the host saved its state throws `IllegalStateException`. `commitAllowingStateLoss()` hides the crash by **silently dropping** that transaction if the process dies. Fix the timing instead: commit only while the lifecycle is at least STARTED.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Back stack and back handling

- `addToBackStack(name)` records the **transaction**; Back reverses it.
- `popBackStack(name, POP_BACK_STACK_INCLUSIVE)` pops back to (and including) a named entry.
- **Multiple back stacks** (bottom navigation): `saveBackStack(name)` / `restoreBackStack(name)` — Navigation uses them for `saveState` / `restoreState`.

```
// Enabled only while there is something to intercept
val discardCallback = requireActivity().onBackPressedDispatcher
    .addCallback(viewLifecycleOwner, enabled = false) {
        showDiscardDialog()
    }

viewLifecycleOwner.lifecycleScope.launch {
    viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
        viewModel.hasUnsavedChanges.collect { unsaved ->
            discardCallback.isEnabled = unsaved
        }
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Back in 2026: predictive back

ANDROID 16 (API 36)

For apps targeting API 36, predictive back animations are on by default, and **`Activity.onBackPressed()` is no longer called**; `KEYCODE_BACK` isn't dispatched. Back logic must live in `OnBackPressedDispatcher` callbacks (or `BackHandler` in Compose).

- Fragment 1.7+ supports predictive back for Fragment transitions with Animators and AndroidX Transitions.
- A callback should be **enabled only when it will actually handle Back —** an always-enabled callback breaks the predictive animation, because the system can't know Back will leave the screen.

INTERVIEW TRAP

"I override `onBackPressed()` in the activity and forward to the current Fragment." — Deprecated since API 33 and ignored with targetSdk 36. Use enabled/disabled dispatcher callbacks.



Q7. Common pitfalls — and the fix

Pitfall	Fix
Binding kept after `onDestroyView` → leak	Set `_binding = null` in `onDestroyView`
Collecting with the Fragment's lifecycle	Use `viewLifecycleOwner` + `repeatOnLifecycle`
`requireContext()` in a callback after detach → crash	Scope work to the view/Fragment lifecycle; cancel on stop
Nested Fragments on the wrong manager	Children: `childFragmentManager`; siblings: `parentFragmentManager`
Keeping Fragment references in a `ViewPager2` adapter	Let `FragmentStateAdapter` create them; talk via ViewModels
RecyclerView adapter outliving the view	Clear `recyclerView.adapter` in `onDestroyView` if the adapter is a property



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. The canonical safe Fragment

```
class OrdersFragment : Fragment(R.layout.fragment_orders) {

    private val viewModel: OrdersViewModel by viewModels()
    private var _binding: FragmentOrdersBinding? = null
    private val binding get() = checkNotNull(_binding)

    override fun onViewCreated(view: View, state: Bundle?) {
        _binding = FragmentOrdersBinding.bind(view)
        val owner = viewLifecycleOwner
        owner.lifecycleScope.launch {
            owner.repeatOnLifecycle(Lifecycle.State.STARTED) {
                viewModel.uiState.collect(::render)
            }
        }
    }

    override fun onDestroyView() {
        _binding = null
        super.onDestroyView()
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 2



Senior

Navigation design · Smooth transitions · Compose interop & migration



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Navigation for a single-activity Fragment app

- **Jetpack Navigation** with a `NavHostFragment` in a `FragmentContainerView`
- **Type-safe routes** (`@Serializable`) with the Kotlin DSL — the modern replacement for XML + Safe Args
- **Nested graphs** per flow (onboarding, checkout) → a natural scope for flow ViewModels
- **Deep links declared on destinations**, so a link builds the correct back stack
- UI talks to a small **navigator interface**, not to `NavController` directly — easier to test and to move to Compose later

```
navController.graph = navController.createGraph(
    startDestination = Home
) {
    fragment<HomeFragment, Home> { label = "Home" }
    fragment<DetailFragment, Detail> {
        deepLink<Detail>(basePath = "https://example.com/item")
    }
}
// Navigate
findNavController().navigate(Detail(itemId = 42))
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Arguments, results & testing

```
@Serializable data object Home
@Serializable data class Detail(val itemId: Long)

class DetailViewModel(handle: SavedStateHandle) : ViewModel() {
    private val itemId = handle.toRoute<Detail>().itemId
}
```

- Arguments are compile-time checked and live in the back-stack entry's saved state → they survive process death.
- Results: previous back-stack entry's `SavedStateHandle` (Q3).
- Test navigation with `TestNavHostController` — assert on `currentBackStackEntry`, no device UI needed.

INTERVIEW TRAP

Navigating twice on a fast double-tap crashes or duplicates screens. Guard with the current destination, or let the ViewModel expose navigation as state that's consumed once.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Smooth transactions and transitions

Keep the main thread free during a transition

- No data loading in `onCreateView` / `onViewCreated` — the ViewModel loads; the Fragment only renders state
- Don't launch `Dispatchers.IO` work from the Fragment and touch views from it — expose main-safe `suspend` functions from the repository

Transitions

- `setReorderingAllowed(true)` everywhere
- `postponeEnterTransition()` until images or lists are ready, then `startPostponedEnterTransition()` — with a timeout so a failed load can't freeze navigation
- Material motion (`MaterialSharedAxis`, `MaterialContainerTransform`) for consistent, GPU-friendly transitions

Lists

- `ListAdapter` (diffing on a background thread), stable IDs where it helps
- `setHasFixedSize(true)` only if item changes never change the RecyclerView's own size



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Measure, don't guess

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?)
{
    postponeEnterTransition(500, TimeUnit.MILLISECONDS)
    binding.list.doOnPreDraw { startPostponedEnterTransition() }
}
```

- **JankStats** in production, tagged with the current screen
- **Macrobenchmark** with `FrameTimingMetric` for "open detail" and "scroll list" journeys in CI
- **Perfetto** traces for long frames: inflation, layout, `RecyclerView` binds
- **StrictMode** in debug builds to catch disk and network access on the main thread



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Compose ↔ Fragment interop

Compose inside a Fragment — dispose with the *view*, not the window:

```
override fun onCreateView(
    inflater: LayoutInflater, container: ViewGroup?, state: Bundle?,
): View = ComposeView(requireContext()).apply {
    setViewCompositionStrategy(
        ViewCompositionStrategy.DisposeOnViewTreeLifecycleDestroyed
    )
    setContent { AppTheme { ProfileRoute(viewModel) } }
}
```

A Fragment inside Compose — `fragment-compose`:

```
@Composable
fun MapPane(modifier: Modifier = Modifier) {
    AndroidFragment<LegacyMapFragment>(modifier.fillMaxSize())
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. A migration strategy that works

1. **Theme first:** one Compose theme that matches the View theme.
2. **Leaf UI:** new components and list items in Compose, hosted in existing Fragments via `ComposeView`.
3. **Screen by screen:** a Fragment becomes a thin shell around a composable screen; ViewModels stay unchanged.
4. **Navigation last:** when most screens are composables, replace the Fragment graph with Compose Navigation. Keep legacy screens with `AndroidFragment` until they're rewritten.

INTERVIEW TRAP

The default strategy disposes the composition when the view is **detached from the window**. A Fragment's view can be detached and re-attached without being destroyed (transitions, some pager setups), so `remember` state is lost. `DisposeOnViewTreeLifecycleDestroyed` ties the composition to the Fragment's **view lifecycle** instead.

FOLLOW-UP

"Why not rewrite everything at once?" — Risk and velocity. Incremental migration keeps shipping features and lets each step be tested and rolled back.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 2

→

Architect

Multi-team navigation · Scoped state · ANRs & responsiveness



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Navigation across many teams and modules



- Each feature exposes **routes and entry points** in a small `:api` module; the UI lives in `:impl`.
- Features depend only on other features' `:api` → no cycles, independent builds.
- Each `:impl` **contributes its graph** to the app via DI multibinding; `:app` assembles the root graph.
- Deep links are declared on the destinations inside each feature and validated in CI.
- Dynamic feature modules only when install size really requires it — they add complexity to builds, testing and navigation.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Contributing graphs with DI

```
// :feature:checkout:api
@Serializable data object CheckoutGraph

// :core:navigation
interface FeatureGraph {
    fun NavGraphBuilder.register()
}

// :feature:checkout:impl
class CheckoutFeatureGraph @Inject constructor() : FeatureGraph {
    override fun NavGraphBuilder.register() {
        navigation<CheckoutGraph>(startDestination = Cart) {
            fragment<CartFragment, Cart> { label = "Cart" }
            fragment<PaymentFragment, Payment> { label = "Payment" }
        }
    }
}

@Module @InstallIn(SingletonComponent::class)
interface CheckoutNavModule {
    @Binds @IntoSet
    fun graph(impl: CheckoutFeatureGraph): FeatureGraph
}
```

`:app` builds the root graph by calling `register()` on every `FeatureGraph` in the injected `Set`. A `FragmentFactory` enables constructor injection for Fragments.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Sharing state in a deep hierarchy

Pick the ViewModel **owner** by asking: *"Who should this state live and die with?"*

Scope	Lives until	Typical state
Fragment (`viewModels()`)	Fragment destroyed	One screen
Parent Fragment	Parent destroyed	Tabs or panes inside one screen
Nested nav graph	Flow popped off the back stack	Multi-step checkout, onboarding
Activity (`activityViewModels()`)	Activity finished	Truly app-wide UI state — rare

```
// Tabs share their parent's ViewModel
private val pane: PaneViewModel by viewModels(
    ownerProducer = { requireParentFragment() }
)

// Every step of checkout shares the flow's ViewModel
private val checkout: CheckoutViewModel by viewModels(
    ownerProducer = {
        findNavController().getBackStackEntry<CheckoutGraph>()
    }
)
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Keeping shared state scalable

- **Narrowest scope that works.** An activity-scoped "god ViewModel" becomes a merge-conflict magnet and a hidden global.
- **Shared ViewModels hold state, not UI logic.** Screens keep their own ViewModels and read the shared one.
- **Model events as state.** A one-off `SharedFlow` event can be lost if no one is collecting; state (e.g. `paymentResult: Result?`) consumed and cleared by the UI is lossless.
- **Persist what matters.** Flow state that must survive process death goes to `SavedStateHandle` (keys) or the repository (data).

INTERVIEW TRAP

Scoping to the activity "because it's easy" leaks flow state into unrelated screens: finish checkout, start it again — and the old cart is still there.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Fragment lifecycle, ANRs & responsiveness

Every lifecycle callback runs on the **main thread**. ANRs happen when it's blocked:

ANR trigger	Limit
Input event not handled	5 s
`BroadcastReceiver.onReceive`	10 s foreground / 60 s background

Android vitals "bad behavior" threshold: **0.47% user-perceived ANR rate** overall, **8%** on any single device model.

- Main-thread work per callback should be **milliseconds**: inflate, bind, start collecting.
- Disk, network, JSON parsing, crypto, image decoding → off the main thread, behind main-safe `suspend` APIs.
- Janky frames (> 16 ms at 60 Hz, > 8 ms at 120 Hz) come long before ANRs — measure them.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Finding ANRs in production

```
val am = context.getSystemService(ActivityManager::class.java)
am.getHistoricalProcessExitReasons(context.packageName, 0, 5)
    .filter { it.reason == ApplicationExitInfo.REASON_ANR }
    .forEach { exit ->
        val trace = exit.traceInputStream
            ?.bufferedReader()?.use { it.readText() }
        crashReporter.reportAnr(
            exit.timestamp, exit.description, trace
        )
    }
```

- `ApplicationExitInfo` (API 30+) gives the **ANR trace from the user's device** on the next launch.
- StrictMode in debug, Android vitals in production, Perfetto for root cause.
- Typical culprits: `SharedPreferences` (`commit()` blocks, and even pending `apply()` writes are flushed on the main thread at `onStop` — migrate to DataStore), database queries in `onViewCreated`, huge layouts inflated on every tab switch.

SENIOR ANSWER IN ONE LINE

"The lifecycle tells you *when* you may touch the UI — never *where* to do the work. Work belongs to the ViewModel and data layer, off the main thread."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 2 · ARCHITECT

Day 2 — cheat sheet

- The Fragment **outlives its view** — use `viewLifecycleOwner` for anything touching views.
- Clear binding in `onDestroyView`. No constructor arguments — use `arguments`.
- Communicate via scoped ViewModels, the Fragment Result API or navigation results — never direct references.
- `add()` keeps the Fragment below RESUMED; prefer `replace` + back stack.
- `setReorderingAllowed(true)`, `FragmentContainerView`, and no `commitAllowingStateLoss` band-aids.
- targetSdk 36: `onBackPressed()` isn't called — use `OnBackPressedDispatcher`.
- Type-safe Navigation routes; deep links on destinations; scope flow state to nested graphs.
- Compose in Fragments: `DisposeOnViewTreeLifecycleDestroyed`. Fragments in Compose: `AndroidFragment`.
- Lifecycle decides *when*, not *where* — heavy work never runs on the main thread.

Day 3 is next → Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700